DATA STRUCTURES ASSESSMENT ANSWERS

Prepared for: **Ameeruddin**

Topics: Data Objects • Dynamic Array Resizing in C • Recursion in File-System Traversal

Assessment structure

Question 1 — 5 marks

Question 2 — 10 marks

Question 3 — 5 marks

Exam-ready notes with examples, C code, and complexity analysis.

1. What is a Data Object? Explain with real-time illustrations (5 marks)

Definition: A data object is a region of memory or a logical entity that stores a value or a collection of related values. It has a type, a value, and usually a name or reference through which a program can access it. The type determines what kind of data it can hold and what operations are valid.

Main characteristics

• **Identity:** It can be distinguished from other objects, for example, a particular student record.

• **Type:** It specifies the permitted data, such as integer, character, array, structure, or object.

• **Value/state:** It contains the data at a particular time; the value may change during program execution.

• **Operations:** Programs can read, update, compare, or process the stored data.

Real-time illustrations

Example	Data object	Explanation
Student information system	A Student record containing roll number, name, and marks	A structure groups related student details and allows the system to store or retrieve one student's record.
Online shopping app	A Cart array containing product IDs and quantities	The app updates quantities, calculates totals, and removes items using the stored data.
Banking application	An account object with account number and balance	Deposits and withdrawals update the balance while the account number identifies the account.
Temperature monitor	A floating-point variable such as temperature = 28.5	The program stores a sensor reading and compares it with a threshold to trigger an alert.

Conclusion: Data objects are the basic units used to represent and manage information in software. Choosing suitable data-object types makes programs easier to organize, process, and maintain.

2. Implement dynamic array resizing in C and analyze time complexity (10 marks)

Introduction: A dynamic array is an array whose storage capacity can grow or shrink while the program runs. In C, memory can be allocated with `malloc()` and resized with `realloc()`. A common efficient strategy is to double the capacity when the array becomes full.

Algorithm

• Start with a small capacity and allocate memory.

• Before inserting an element, check whether size equals capacity.

• If full, double the capacity and call realloc() to request a larger memory block.

• If realloc() fails, retain the original pointer and report an allocation error.

• Insert the new element and increase size.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    size_t size;
    size_t capacity;
} DynamicArray;

int initArray(DynamicArray *a) {
    a->size = 0;
    a->capacity = 2;
    a->data = malloc(a->capacity * sizeof *a->data);
    return a->data != NULL;
}

int append(DynamicArray *a, int value) {
    if (a->size == a->capacity) {
        size_t newCapacity = a->capacity * 2;
        int *temp = realloc(a->data, newCapacity * sizeof *a->data);
        if (temp == NULL) {
            return 0;  /* Original allocation remains valid */
        }
        a->data = temp;
        a->capacity = newCapacity;
    }

    a->data[a->size] = value;
    a->size++;
    return 1;
}

void freeArray(DynamicArray *a) {
    free(a->data);
    a->data = NULL;
    a->size = 0;
    a->capacity = 0;
}

int main(void) {
    DynamicArray a;

    if (!initArray(&a)) {
        fprintf(stderr, "Initial allocation failed.\\n");
        return 1;
    }

    for (int i = 1; i <= 10; i++) {
        if (!append(&a, i * 10)) {
            fprintf(stderr, "Could not grow the array.\\n");
            freeArray(&a);
```

```
            return 1;
        }
    }

    for (size_t i = 0; i < a.size; i++) {
        printf("%d ", a.data[i]);
    }
    printf("\\nSize = %zu, Capacity = %zu\\n", a.size, a.capacity);

    freeArray(&a);
    return 0;
}
```

Explanation of the program

• `size` records how many elements are stored; `capacity` records how many elements fit before resizing is needed.

• `realloc()` may extend the existing block or move it to a new location, so its result is first stored in a temporary pointer.

• The array doubles from capacity 2 to 4, 8, 16, and so on. Ten elements therefore fit in a capacity of 16.

Time-complexity analysis

Operation	Time complexity	Reason
Access by index	O(1)	The element is accessed directly by its index.
Append when space exists	O(1)	Write the element at the next free position.
Append when resizing is required	O(n)	Existing elements may need to be copied to a larger memory block.
Append, amortized	O(1)	With capacity doubling, expensive resizes are spread over many insertions.
Space complexity	O(n)	The array stores n elements and may reserve extra capacity.

Conclusion: Dynamic arrays provide flexible storage. Doubling capacity gives O(1) amortized time per append, although an individual resize can take O(n) time. Always check allocation failures and free allocated memory when it is no longer required.

3. Discuss the use of recursion in file-system traversal (5 marks)

Definition: Recursion is a programming technique in which a function calls itself to solve smaller instances of the same problem. A file system is naturally hierarchical: a directory may contain files and other directories. This structure makes recursion useful for visiting every item.

How it works

• Begin at a starting directory.

• List each entry in that directory.

• If an entry is a file, process or display it.

• If an entry is a subdirectory, call the traversal function for that subdirectory.

• Stop when a directory has no more entries to visit; this is the base/termination condition.

Pseudocode

```
TRAVERSE(directory):
    for each entry in directory:
        if entry is a file:
            display entry
        else if entry is a subdirectory:
            TRAVERSE(entry)
```

Real-time example

Suppose a folder named **College** contains **Notes.pdf** and a subfolder named **Programming**. The Programming folder contains **arrays.c** and another folder named **Projects**. A recursive traversal displays Notes.pdf, enters Programming, displays arrays.c, then enters Projects and processes its contents. The same function works at every directory level.

Advantages and limitations

• **Advantages:** Simple code, naturally matches the directory tree, and handles nested folders without knowing the depth in advance.

• **Limitations:** Very deep directory trees can use a large call stack. Real systems must handle permission errors, symbolic links/cycles, and inaccessible directories.

> **Complexity:** If there are V files and directories and each entry is processed once, traversal takes O(V) time. The recursion stack uses O(h) extra space, where h is the maximum directory depth.

Conclusion: Recursion is a clear and effective method for traversing hierarchical file systems because each subdirectory can be handled by the same function.